Fusion 360 MCP

Model parts and assemblies in Autodesk Fusion 360 by talking to Claude.

You describe what you want — "a 60×40×20 mm enclosure, 2 mm walls, four M3 tapped holes in the corners" — and Claude builds it in Fusion, looks at the result with a screenshot, checks the dimensions, and fixes its own mistakes. It can also export STEP/STL/DXF and save versioned designs.

This is a hobbyist project shared as-is. It works well for the bread-and-butter of mechanical CAD (boxes, cylinders, holes, fillets, shells, tapped holes, multi-part assemblies with joints) and hands off to raw Fusion API code for the exotic stuff.

What you get

About 60 tools Claude can call, grouped as:

- **Primitives & features** — boxes, cylinders, holes, fillets, shells, tapped holes (with correct tap-drill sizing for hand-tapping). Cutting tools auto-report volume change and warn on no-op cuts.
- **Parametric** — named user parameters so models stay adjustable.
- **Faces** — sketch and drill on a selected face, not just the base plane.
- **Assemblies** — components, occurrences, linear copies of hardware, grounding, and rigid / revolute joints.
- **Verification & diagnostics** — return a screenshot to Claude, measure real distances, ASCII cross-sections (solid/void scans), face/edge listings with stable IDs, and a rich one-call model-state report.
- **Async & resilience** — long operations run as pollable jobs with incremental progress; liveness/state stay responsive even during a hung op; the listener refuses known host-crashing API calls.
- **Document safety** — every state read shows the active document name + lineage; saves report which lineage they wrote; save-as-new and read-only version listing guard against overwriting the wrong design.
- **Export & save** — STEP, STL (3D print), DXF (laser/waterjet), and save / open / list designs across sessions.
- **Escape hatch** — `run_fusion_code` / `run_fusion_code_safe` run arbitrary Fusion API Python (with injected unit/observability helpers and documented sketch-plane mapping), the latter with automatic rollback on error.

Full list: `docs/TOOLS.md`.

How it works (the 30-second version)

```
You (chat) → Claude Desktop → fusion_mcp_server.py → localhost:8765 → MCPListener (add-in
inside Fusion)
                                                                                    |
        Claude ←— text / tracebacks / screenshots ←————————————————————————————————┘
```

Two long-running pieces:

1. **MCPListener** — a small add-in that runs *inside Fusion* and executes modeling commands on Fusion's main thread.
2. **fusion_mcp_server.py** — an MCP server that **Claude Desktop launches for you** in the background; it relays Claude's tool calls to the listener.

You don't start the server yourself — Claude Desktop does, based on its config. You just keep Fusion open with the add-in running.

Quick start

1. **Install** — follow `docs/INSTALL.md`. It covers the add-in, the Python deps, and the Claude Desktop config, with fixes for the common snags.
2. **Check your setup** — run the preflight: - Windows: double-click `launch.bat` - macOS/Linux: `./launch.sh` It tells you exactly what's missing.
3. **Confirm it's live** — in Claude Desktop, ask: **"Ping Fusion."** A reply of "alive" means everything's connected.
4. **Build something** — try:

   > "Make a 20 mm cube and show me."

then the real shakedown:

> "Make a 60×40×20 mm parametric box, shell it to 2 mm with an open top, add an M3 tapped hole 5 mm in from each top corner, and show me top and iso views."

Requirements

- Autodesk Fusion 360 (current version).
- Claude Desktop (the desktop app — this uses local MCP over stdio).
- Python 3.10+ (3.10–3.12 are the safest; see INSTALL note on 3.13/3.14).

Files in this package

File	What it is
`MCPListener.py`	The add-in that runs **inside Fusion**.
`fusion_mcp_server.py`	The MCP server **Claude Desktop launches**.
`launch.py` / `launch.sh` / `launch.bat`	Preflight health-checker.
`docs/claude_project_instructions.md`	Paste into a Claude Project to teach it the conventions.
`requirements.txt`	Python dependencies.
`docs/INSTALL.md`	Step-by-step install + troubleshooting.
`docs/pdf/`	Print-ready PDFs of every doc (plus `ALL-DOCS` combined).
`docs/render_pdfs.py`	Regenerates those PDFs after you edit the markdown.
`docs/TOOLS.md`	Reference for every tool.
`docs/ARCHITECTURE.md`	How the pieces fit, for the curious / for hacking.
`LICENSE`	MIT.

A few honest caveats

- The listener executes arbitrary Python sent to **localhost only** (`127.0.0.1:8765`). That's required for it to work, but don't expose that port to a network. See ARCHITECTURE for the security model.
- The canned tools assume principal planes and box-like geometry for face selection. Complex/organic shapes use the raw-code escape hatch — Claude knows when to switch.
- This isn't an official Autodesk or Anthropic product. It's glue code. Treat early runs as a shakedown and keep your work saved.

Happy modeling.